



POSIX Threads and OpenMP

058165 - PARALLEL COMPUTING

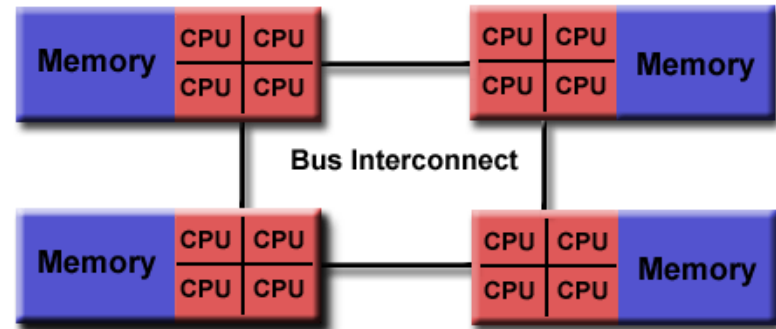
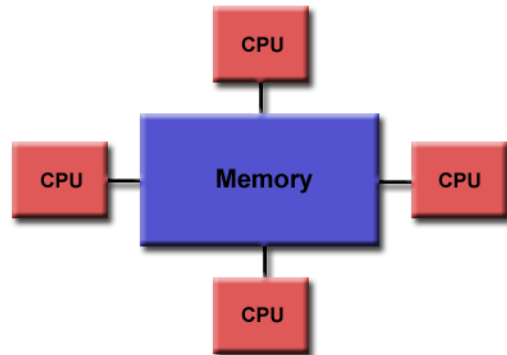
Fabrizio Ferrandi

Politecnico di Milano

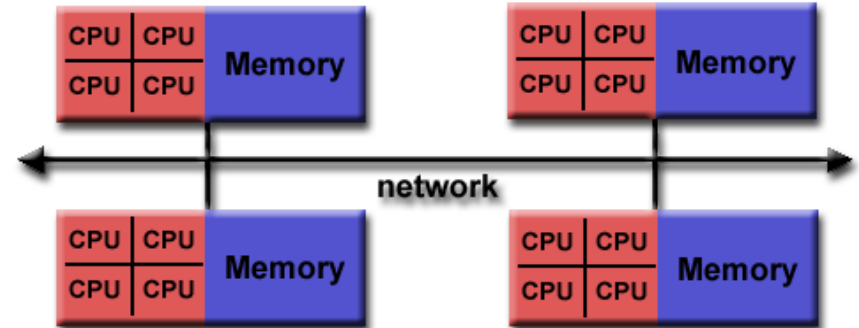
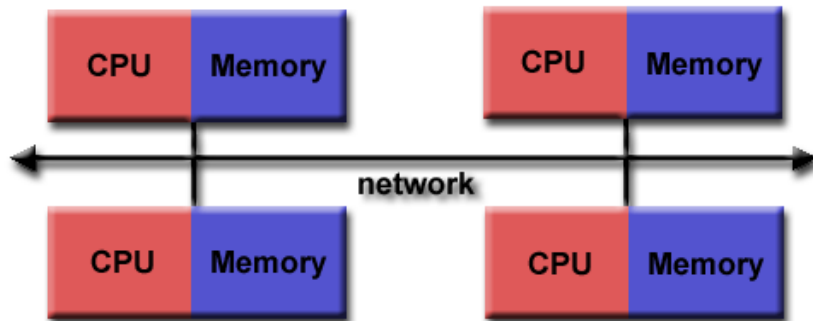
Dipartimento di Elettronica, Informazione e Bioingegneria

fabrizio.ferrandi@polimi.it

- ❑ POSIX threads and OpenMP are two implementations of a **shared memory** *parallel programming model* with **threads**
- ❑ Parallel patterns describe an *algorithm*
- ❑ Parallel architectures describe *computing resources, memories and interconnections*
- ❑ Parallel programming models are an *abstraction* above the hardware
- ❑ There are different *implementations* for each parallel programming models



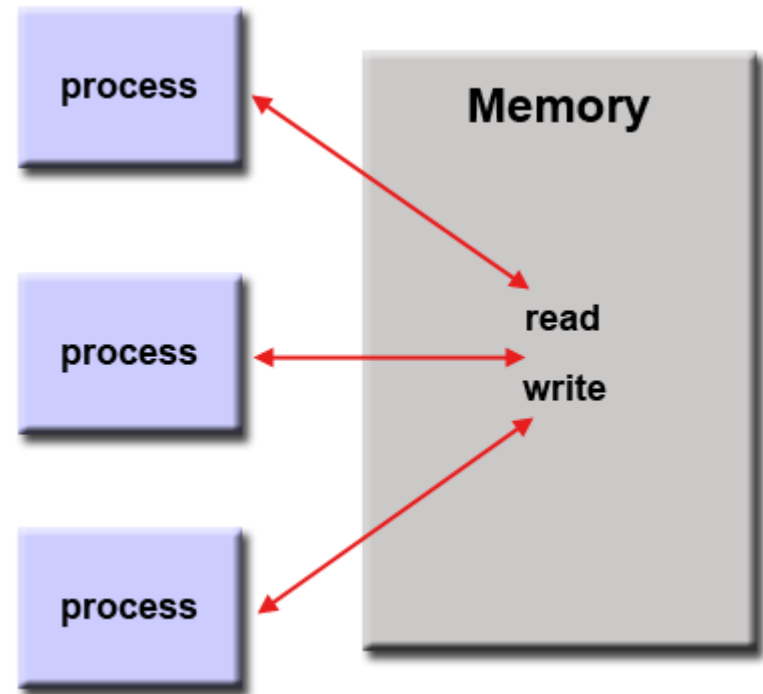
□ Shared memory



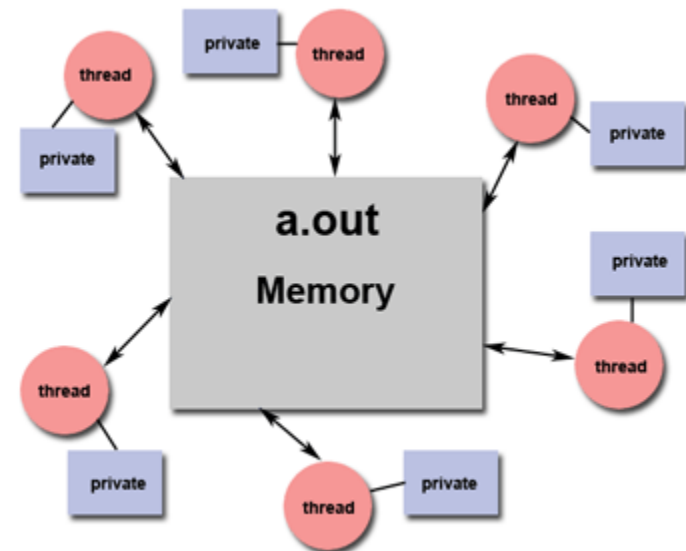
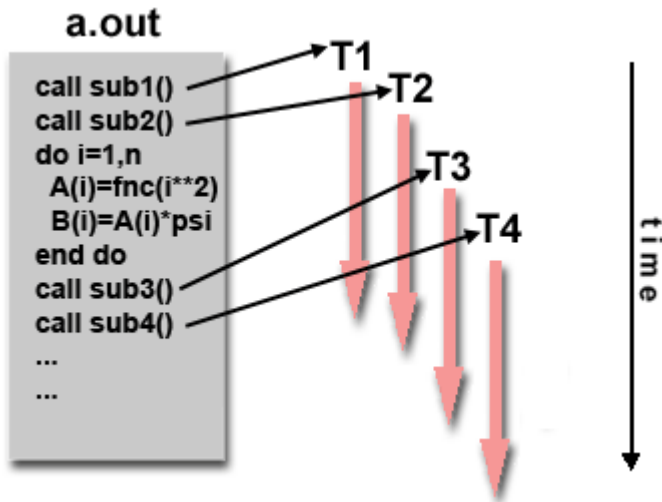
□ Distributed memory

- ❑ Shared memory (without threads)
 - ❑ **Threads**
 - ❑ Message Passing / Distributed Memory
 - ❑ Data Parallel
 - ❑ Single Program Multiple Data
 - ❑ Multiple Program Multiple Data
-
- ❑ They *may or may not* correspond to the underlying hardware architecture

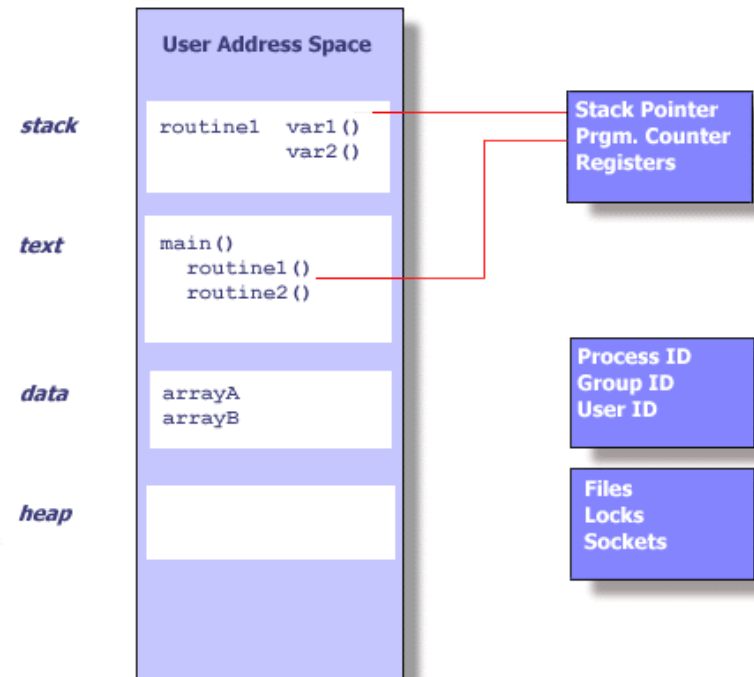
- ❑ Simplest model
- ❑ Processes share a common address space
 - ❑ same physical address space, different virtual address space per-process.
- ❑ No explicit communication
- ❑ Difficult to maintain locality



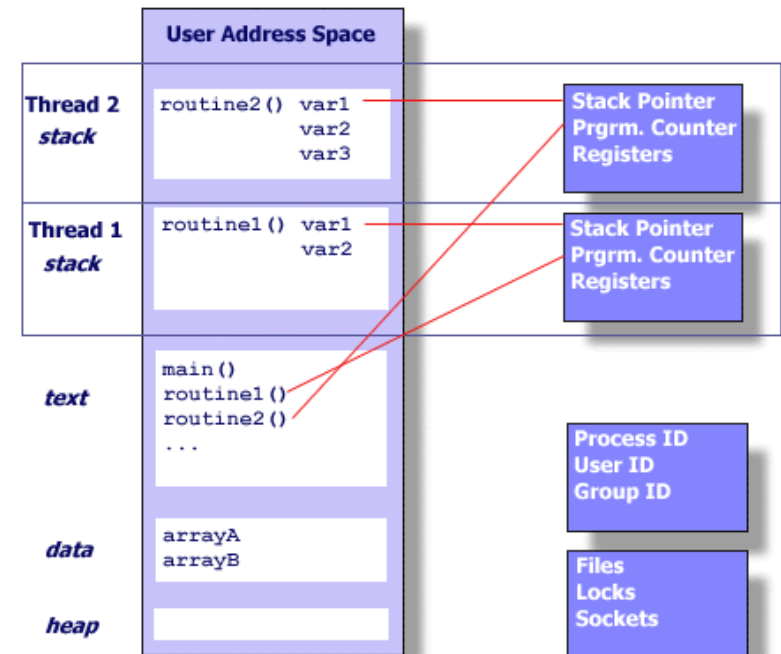
- ❑ Single process with multiple threads that can execute concurrently
- ❑ Each thread has local data, but it can access all resources acquired by the main process



- ❑ A UNIX **process** is created by the operating system
- ❑ Processes contain information about program resources and program execution state
 - ▶ Process ID, process group ID, user ID, and group ID,
 - ▶ Environment
 - ▶ Working directory
 - ▶ Program instructions
 - ▶ etc.



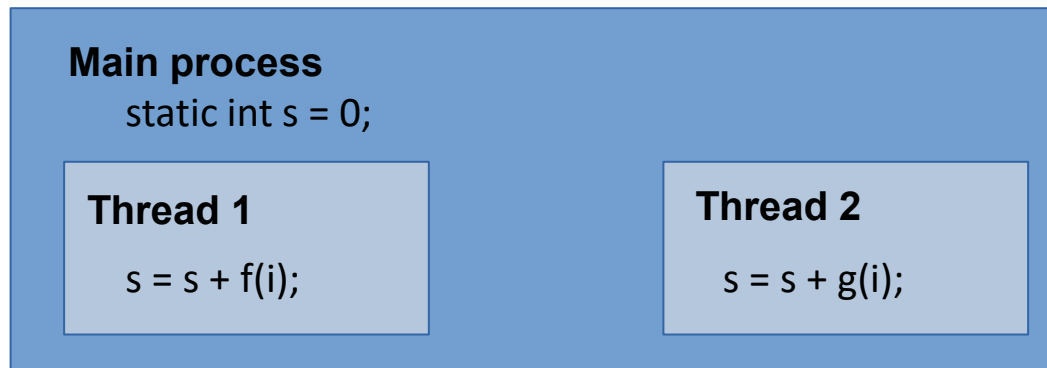
- ❑ A **thread** is an independent stream of instructions within a process
- ❑ Threads can be scheduled to run by the operating system
- ❑ Any thread can execute at the same time as other threads
- ❑ Threads have local resources and can access the shared process resources



- ❑ A thread can be seen as any "procedure" that runs independently from its main program
- ❑ Example of multi-thread program:
 - ▶ `main` which calls some procedures
 - ▶ all the procedure run **simultaneously and/or independently** by the operating system
- ❑ Threads can be created *dynamically* during execution
- ❑ Duplicate only the *bare essential* resources
 - ▶ Multi-thread is "lighter" than multi-processes

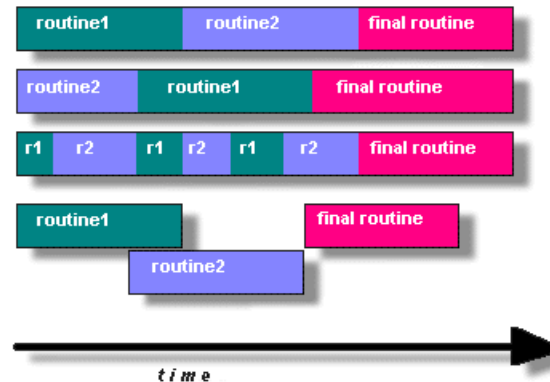
- ❑ Threads exists within a process and share most of the process resources
 - ▶ Changes made by one thread to shared system resources (such as closing a file) will be seen by all other threads
 - ▶ Two pointers having the same value point to the same data
 - ▶ **Implicit communication** by reading and writing shared variables
 - ▶ Reading and writing to the same memory locations requires **explicit synchronization** by the programmer

- ❑ **Data race** or **race condition** occurs when
 - ▶ Two threads (or processors) access the same variables, and at least one does a write
 - ▶ Accesses are concurrent, so they could happen at the same time

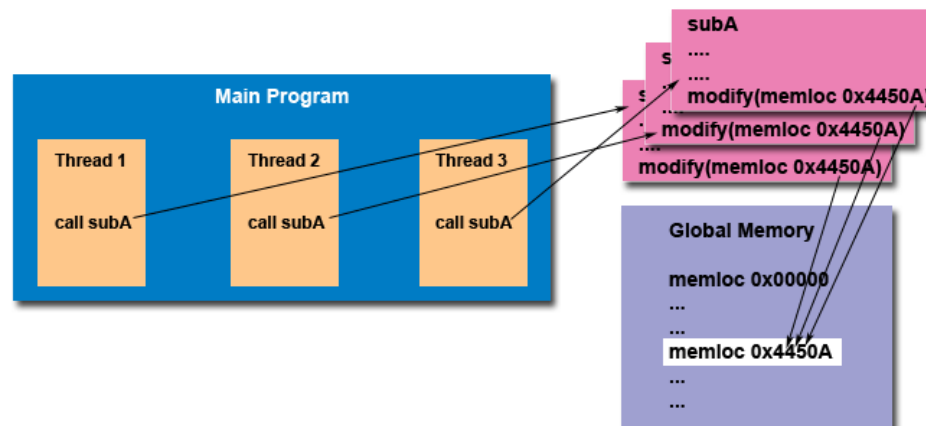


- ❑ Race condition on the global variable `s` → **synchronization** is needed

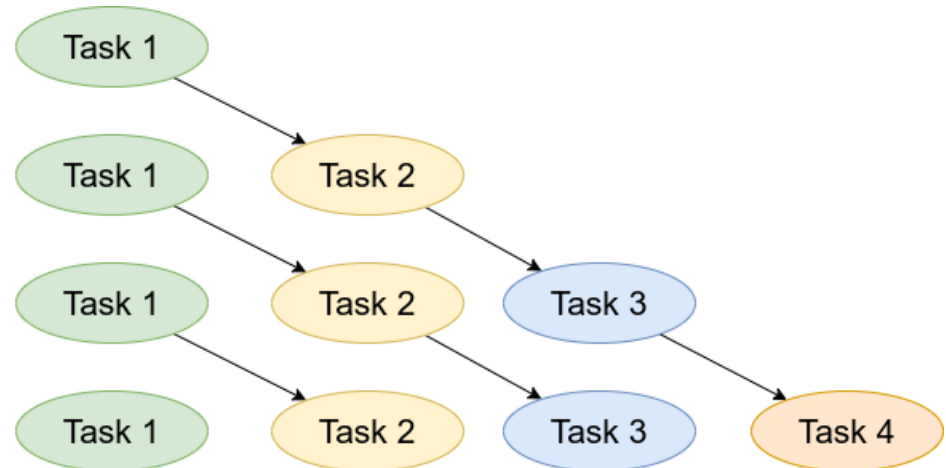
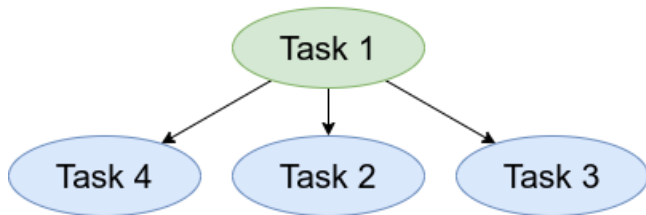
- ❑ Can my program be implemented with threads?
 - ▶ Can it be organized in discrete, *concurrent* tasks?



- ▶ Is it *thread-safe*?



- ❑ Common models for threaded programs:
 - ▶ Manager/worker
 - ▶ Pipeline



- ❑ The programmer is responsible for handling parallelism and synchronization, usually through
 - ▶ A library of subroutines
 - ▶ A set of compiler directives
- ❑ Historically, hardware vendors have implemented their **own proprietary versions** of threads
- ❑ We will see two different **standards**:
 - ▶ POSIX Threads (Pthreads)
 - ▶ OpenMP

- ❑ IEEE POSIX 1003.1c standard (1995) specifies the **API to *explicitly* manage threads**
- ❑ Standard is (slightly) evolving:
 - ▶ IEEE Std 1003.1-2017/Open Group Base Specification Issue 7
- ❑ API is a set of *C language* programming types and procedure calls
 - ▶ Header file: `pthread.h`
 - ▶ Library (stand-alone or part of other library)
 - ▶ Add `-pthread` to the gcc options on Linux

❑ Thread management

- ▶ Creation
- ▶ Joining
- ▶ Detaching
- ▶ Set/query attributes

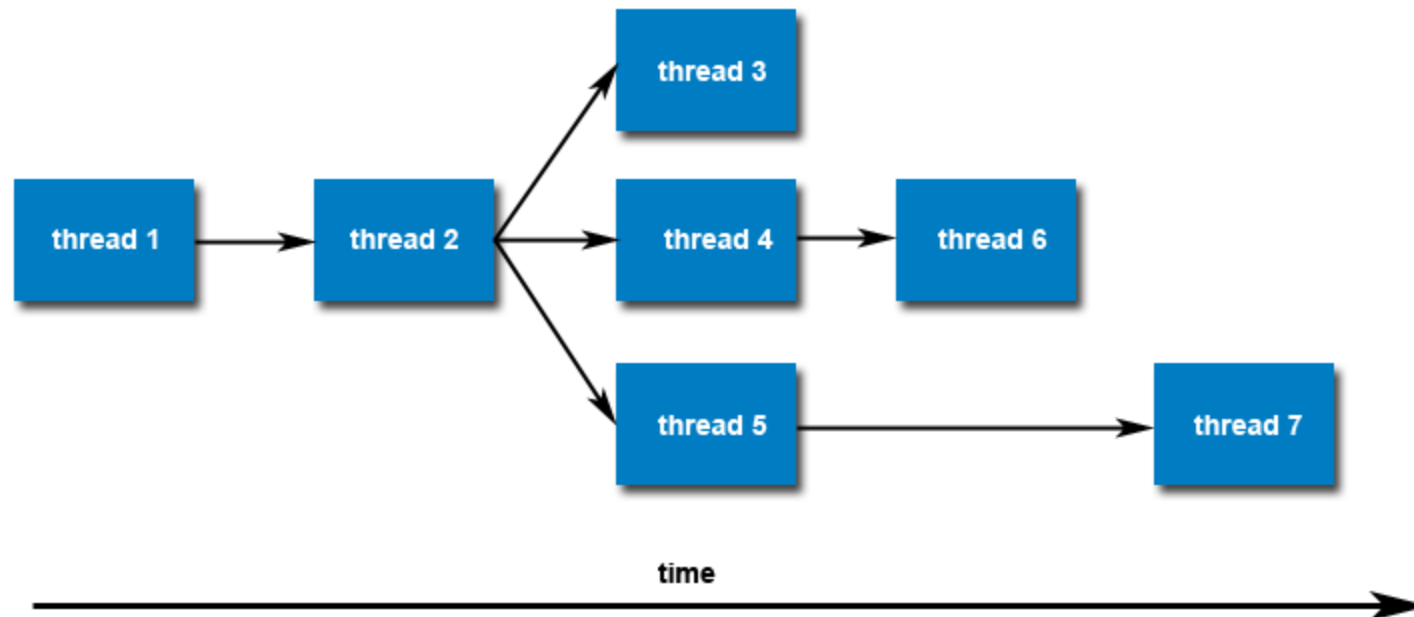
❑ Synchronization

- ▶ Mutexes: Control access to mutual exclusion sections
- ▶ Condition variables: Control conditional inter-threads communications

Thread Management: Creation

17

- ❑ Once created, threads are peers, and may create other threads: there is no implied hierarchy or dependency between threads.
- ❑ The maximum number of threads depends on particular implementation



```
int pthread_create(pthread_t * thread, const pthread_attr_t * attr,  
void * (* start_routine) (void *), void *arg)
```

- ❑ `thread`: identifier for the new thread returned by the subroutine
- ❑ `attr`: used to set thread attributes
 - ▶ Joinable/detached
 - ▶ Scheduling
 - ▶ Stack size
- ❑ `start_routine`: the C routine that the thread will execute once it is created.
- ❑ `arg`: argument passed to `start_routine`. It must be passed by address as a pointer cast of type void.

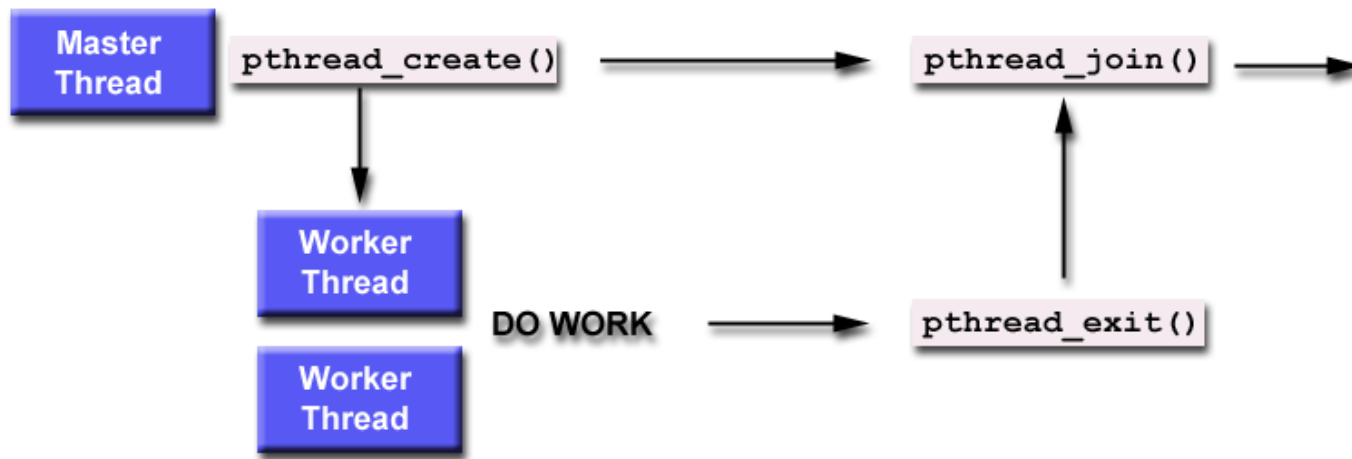
- ❑ The thread returns from its starting routine
- ❑ The thread makes a call to the `pthread_exit` subroutine
- ❑ The thread is canceled by another thread via the `pthread_cancel` routine
- ❑ The entire process is terminated
- ❑ Created threads survive if main calls `pthread_exit`

Thread Management: Joining

20

```
int pthread_join(pthread_t thread, void ** retval)
```

- ❑ `pthread_join` blocks the calling thread until the specified thread terminates
- ❑ `retval` will contain the copy of the argument of `pthread_exit`



- ❑ Threads can be created as joinable or detached
- ❑ A joinable thread can become detached (not vice versa)
- ❑ **Explicitly set** joinable or detached attribute
 - ▶ Default attribute should be joinable
 - ▶ Not all implementations adopt this default
 - ▶ Detached threads may consume less resources

Thread Management:

Joining through Barriers

22

```
int pthread_barrier_init(pthread_barrier_t * barrier,  
    pthread_barrierattr_t * attr, unsigned int count)
```

```
int pthread_barrier_wait(pthread_barrier_t * barrier)
```

- ❑ `pthread_join` requires to wait for a specific thread exit
- ❑ Several threads can be synchronized among them through **barriers**
 - ▶ all the threads of the group need to wait for all the other threads, then they can resume working
- ❑ `count` is the number of threads to be waited
- ❑ Not all implementations include barriers!

- ❑ Mutex (*mutual exclusion*) variables are the basic method to protect shared data when multiple writes occur
- ❑ Only one thread can **lock** a mutex variable at any given time
- ❑ If several threads try to lock a mutex only one thread will be successful

```
pthread_mutex_lock(&my_lock);  
    /* critical section */  
pthread_mutex_unlock(&my_lock);
```

- ❑ Threads that could not acquire the mutex are blocked

- ❑ Less blocking overhead: `trylock`
 - ▶ The thread can keep on working while the lock is not available
- ❑ Multiple mutexes can lead to **deadlocks**

Thread 1

```
lock(a);  
lock(b);
```

Thread 2

```
lock(b);  
lock(a);
```

- ❑ Three types of mutexes:
 - ▶ Normal
 - ▶ Recursive
 - ▶ Error check

Synchronization: Condition Variables

25

- ❑ Mutexes implement synchronization by serializing data accesses
- ❑ **Condition variables** allow threads to synchronize *explicitly* by signaling the meeting of a condition
- ❑ Without condition variables, the programmer would need to *poll* to check if the condition is met

Thread 1

```
lock(a);  
wait on(cond, a);  
do work;  
unlock(a);
```

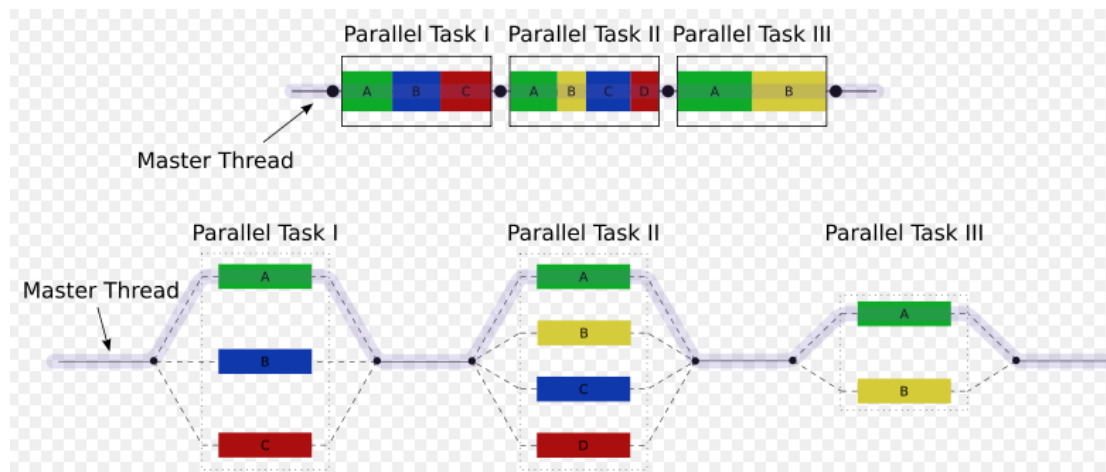
Thread 2

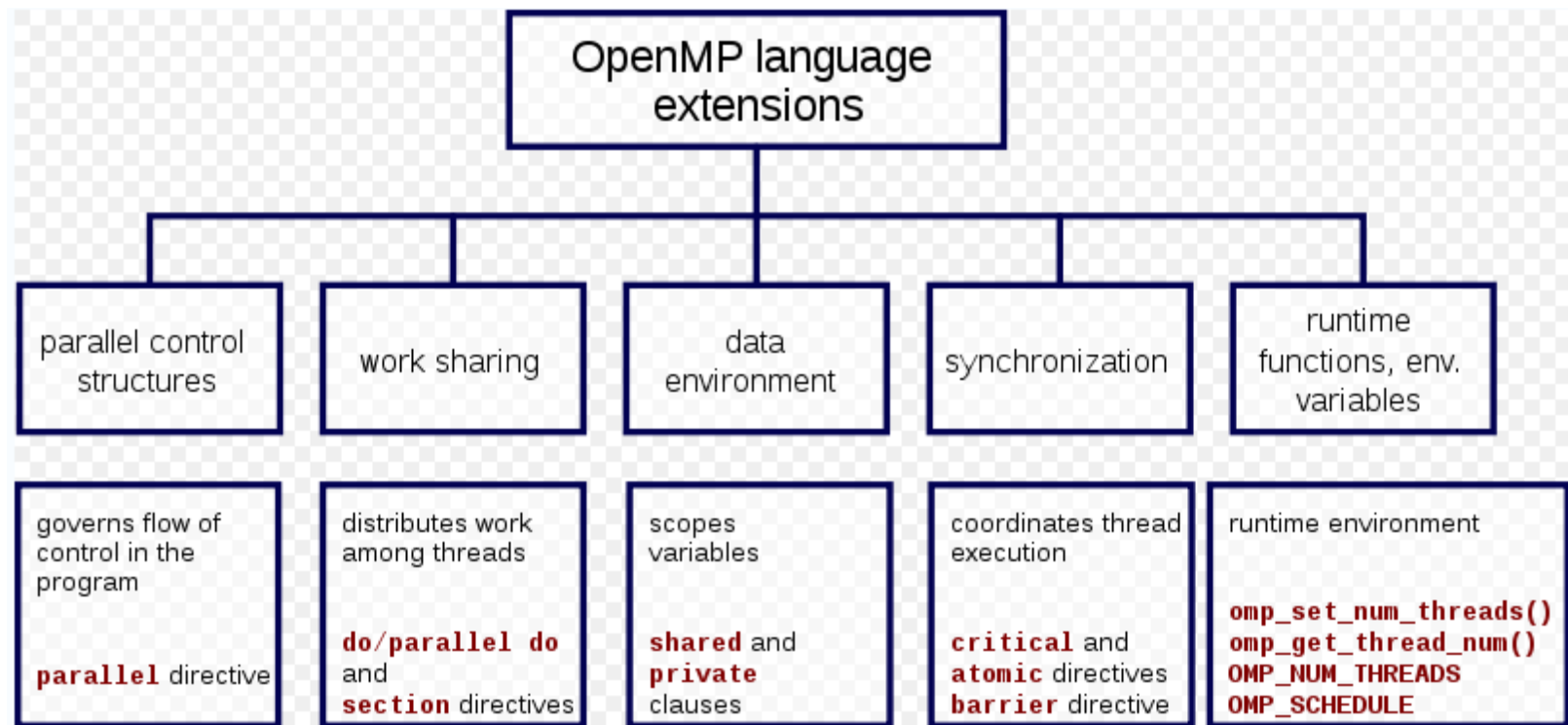
```
lock(a);  
do work;  
signal(cond);  
unlock(a);
```

- ❑ OpenMP (Open Multi-Processing) is an API for multi-threaded shared memory programming
 - ▶ Provides **compiler directives** (~80%), library routines (~19%), and environment variables (~1%)
 - ▶ Needs compiler support
 - ▶ Header `omp.h`
 - ▶ Add `-fopenmp` to the gcc options on Linux
- ❑ Evolving standard (first specification in 1997, version 5.1 in 2020)

- ❑ Standardization and portability
 - ▶ Standard across a variety of shared memory architectures and platforms
 - ▶ Supports Fortran, C and C++
 - ▶ Scalable from embedded systems to the supercomputer
- ❑ Ease of use
 - ▶ *Simple* and *limited* set of directives → 3 or 4 are enough to implement significant parallelism
 - ▶ *Incremental* parallelization of a serial program
 - ▶ Coarse-grained and fine-grained parallelism

- ❑ OpenMP is based on the fork-join paradigm:
 - ▶ A *master* thread forks a specified number of *slave* threads
 - ▶ Tasks are divided among slaves
 - ▶ Slaves run concurrently as the runtime environment allocates threads to different processors





- ❑ Preprocessor directives called **pragma** (pragmatic information).
 - ▶ They are the main part of the OpenMP standard
 - ▶ Identify tasks and their synchronizations

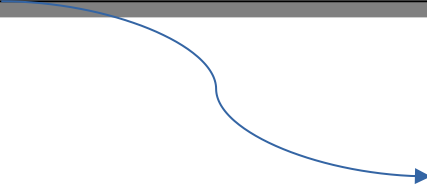
```
#pragma omp <name> [list of clauses]
```
- ❑ Auxiliary C functions
 - ▶ Used to set and get relevant information such as number of available threads
 - ▶ Used to manage explicit locks
- ❑ Environment variables

```
#pragma omp parallel
{
    /* parallel section */
}
```

- ❑ OpenMP programs execute serially until they reach a `parallel` directive
 - ▶ The thread that was executing the code spawns a group of slave threads and becomes the master (thread ID 0)
 - ▶ The code in the structured block is replicated, each thread executes a copy
 - ▶ At the end of the block there is an implied barrier, only the master thread continues

- ❑ Optional clauses to the `parallel` directive:
 - ▶ Conditional parallelization with `if(condition)`
 - ▶ Number of spawned threads with `num_threads(int)`
 - ▶ Data scope clauses (see later)
- ❑ Under the hood, your compiler might replace the directive with a Pthreads implementation

```
#pragma omp parallel num_threads (8)
```



```
for (i = 0; i < 8; i++)  
    pthread_create(..  
    .);  
for (i = 0; i < 8; i++)  
    pthread_join(...);
```

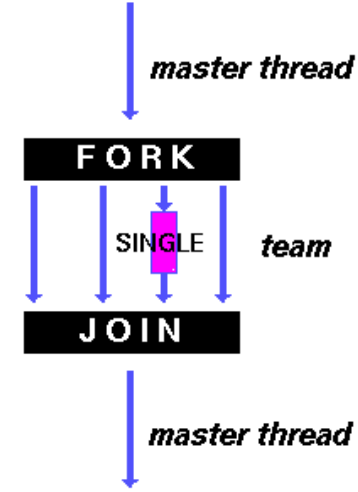
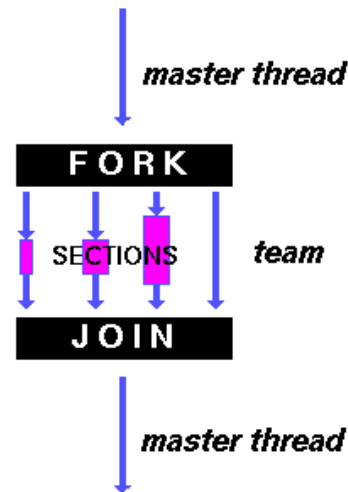
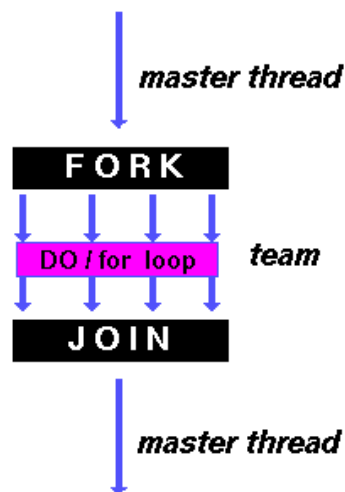

- ❑ The number of threads in a parallel region is determined by the following factors, in order of precedence:
 - ▶ Evaluation of the `if` clause
 - ▶ Value of the `num_threads` clause
 - ▶ Use of the `omp_set_num_threads()` library function
 - ▶ Setting of the `OMP_NUM_THREADS` environment variable
 - ▶ Implementation default, e.g., the number of CPUs on a node.

- ❑ Work-sharing constructs divide the execution of a code region among the members of the team that encounter it
 - ▶ A work-sharing construct must be *enclosed within* a `parallel` region in order for the directive to execute in parallel
 - ▶ Work-sharing constructs do not launch new threads
 - ▶ There is no implied barrier upon entry to a work-sharing construct, however there is an implied barrier at the end of a work sharing construct

for. shares iterations of a loop across the team (data parallelism)

sections: breaks work into separate, discrete sections, each executed by a thread (functional parallelism)

single/master. serializes a section of code.



```
#pragma omp parallel
{
    #pragma omp for
    /* for loop */
}
```

❑ Parallelize execution of iterations

- ▶ Iterations number cannot be internally modified

❑ Possible clauses include:

- ▶ `schedule` describing how iterations of the loop are divided among the threads in the team
- ▶ `nowait` to avoid synchronizing at the end of the parallel loop.
- ▶ Data-scope clauses (see later)

□ Most typical `for` schedules

- ▶ `static`: loop iterations are divided into blocks of size `chunk` and then statically assigned to threads. If `chunk` is not specified, the iterations are evenly (if possible) divided contiguously among the threads.
- ▶ `dynamic`: loop iterations are divided into blocks of size `chunk`, and dynamically scheduled among the threads; when a thread finishes one chunk, it is dynamically assigned another. The default chunk size is 1.
- ▶ `runtime`: depends on the environment variable `OMP_SCHEDULE`

- ❑ A reduction variable in a loop aggregates (e.g., accumulates) a value that:
 - ▶ depends on each iteration of the loop
 - ▶ does not depend on the iteration order
- ❑ The `reduction (operator: list)` clause helps to perform a reduction
 - ▶ A private copy of each variable in the list is updated by each thread
 - ▶ At the end the reduction operation is applied to all private copies and the end result is written into a global variable
 - ▶ Available operators: `+` `*` `-` `&` `|` `^` `&&` `||` `min` `max`

```
#pragma omp parallel
{
    #pragma omp sections
    {
        #pragma omp section
        {
            /* code section 1 */
        }
        #pragma omp section
        {
            /* code section 2 */
        }
    }
}
```

- ❑ Specifies that the enclosed section(s) of code are to be executed in parallel
- ❑ Each section is executed once by a thread in the team

```
#pragma omp parallel
{
    #pragma omp single
    {
        /* code section */
    }
    #pragma omp master
    {
        /* code section */
    }
}
```

- ❑ `single` specifies that a section of a code is executed only by a single thread
- ❑ `master` specifies that a section of a code is executed only by the master


```
#pragma omp task
{
    /* code section */
}
#pragma omp taskwait
#pragma omp taskyield
```

- ❑ The `task` directive specifies a work unit which may be executed or deferred to another thread in the same team
- ❑ The `taskwait` directive introduces a barrier
- ❑ The `taskyield` directive interrupts execution of the current task
 - ▶ May be resumed by the same thread (tied clause) or by another (untied)

```
#pragma omp critical [name]
{
    /* code section */
}
```

- ❑ The `critical` directive specifies a region of code that must be executed by only one thread at a time
- ❑ The optional `name` enables multiple different critical regions:
 - ▶ names act as global identifiers: different critical regions with the same name are treated as the same region
 - ▶ all critical sections which are unnamed, are treated as the same section

```
#pragma omp barrier
```

- ❑ The `barrier` directive synchronizes all threads in the team
- ❑ When a `barrier` directive is reached, a thread will wait at that point until all other threads have reached that barrier. All threads then resume executing in parallel the code that follows the barrier
- ❑ Not very used because of implicit synchronization of other constructs

```
#pragma omp atomic  
/* statement */
```

- ❑ The `atomic` directive ensures that a specific storage location is accessed atomically
- ❑ Multiple reads and writes are not allowed
- ❑ Only valid for the following statement, not for a structured block

- ❑ OpenMP is based upon the shared memory programming model so most variables are **shared by default**
- ❑ The OpenMP Data Scope Attribute Clauses are used to explicitly define how variables should be scoped. They include:
 - ▶ `private`
 - ▶ `shared`
 - ▶ `default`
 - ▶ `reduction`

- ❑ Data Scope Attribute Clauses are used in conjunction with several directives to
 - ▶ define how and which data variables in the serial section of the program are transferred to the parallel sections
 - ▶ define which variables will be visible to all threads in the parallel sections and which variables will be privately allocated to all threads

```
#pragma omp <name> private (list)
```

- ❑ The `private` clause declares variables in its list to be private to each thread
- ❑ `private` variables behave as follows:
 - ▶ a new object of the same type is declared once for each thread in the team
 - ▶ all references to the original object are replaced with references to the new object
 - ▶ assume that each variable is uninitialized for each thread – or use `firstprivate`

```
#pragma omp <name> shared (list)
```

- ❑ The `shared` clause declares variables in its list to be shared among all threads in the team
- ❑ A `shared` variable exists in only one memory location and all threads can read or write to that address
- ❑ It is the programmer's responsibility to ensure that multiple threads properly access `shared` variables


```
#pragma omp <name> default (shared | none)
```

- ❑ The scope of *all* variables is set to `shared` or `none`
 - ▶ `default (shared)` is already the default, so it can be omitted
- ❑ Specific variables can be exempted from the default using specific clauses (`private`, `shared`, etc.)
- ❑ Using `none` as a default requires that the programmer explicitly scope all variables
- ❑ In some implementations `default (private)` is also an option

- ❑ The OpenMP standard defines an API for library calls that perform a variety of functions to control execution of the program
- ❑ `int omp_get_num_threads()`
 - ▶ Returns the number of threads that are currently in the team executing the parallel region from which it is called
- ❑ `int omp_get_thread_num()`
 - ▶ Returns an identifier for the thread making this call. This number will be between 0 and `omp_get_num_threads - 1`. The master thread of the team is thread 0

- ❑ `void omp_set_num_threads(int num_threads)`
 - ▶ Sets the number of threads that will be used in the next parallel region
- ❑ `double omp_get_wtime()`
 - ▶ Provides a wall clock timing routine
- ❑ `double omp_get_wtick()`
 - ▶ Returns the number of seconds between two clock ticks

- ❑ Environment variables are set to control execution of all programs, for example with the `export` command in `bash`
- ❑ `OMP_SCHEDULE`
- ❑ `OMP_NUM_THREADS`
- ❑ `OMP_NESTED`
- ❑ `OMP_STACKSIZE`
- ❑ `OMP_WAIT_POLICY`
- ❑ `OMP_PROC_BIND`

- ❑ *Introduction to Parallel Computing, POSIX Threads Programming and OpenMP Tutorial* by Blaise Barney, Lawrence Livermore National Laboratory
<https://hpc.llnl.gov/training/tutorials>
- ❑ UC Berkeley CS267: *Applications of Parallel Computers*
<https://sites.google.com/lbl.gov/cs267-spr2020>
- ❑ “Introduction to Parallel Computing”, Ananth Grama, Anshul Gupta, George Karypis, Vipin Kumar <https://www-users.cs.umn.edu/~karypis/parbook/>
- ❑ Video series from Tim Mattson (Intel) [Introduction to OpenMP](#)
- ❑ Slides prepared by Serena Curzel and Marco Lattuada for previous editions of this course